
APS360 Project Final Report

Introduction

Living in Toronto, there are a lot of vehicle license plate recognition systems: from Green P Parking to U of T Mississauga campus parking [1]. These systems have significantly enhanced the efficiency of urban operations and have become an indispensable part of the city's infrastructure. Inspired by these systems, the goal of my project is to develop a deep learning model specifically designed to detect and recognize Ontario license plates. I'm interested in this as it is used in daily life and barely makes mistakes. Deep learning is the most reasonable approach for this problem because it's a camera-based technology, and traditional computer algorithms can't recognize license plates under irregular or extreme lighting conditions or angles, and it's hard to fully describe all the features of license plates for traditional machine learning. In comparison, deep learning doesn't need to be explicitly told what edges, shapes, and colours look like; it automatically learns the features within its multi-layered neural networks, which is the most effective and reasonable approach for this project.

Illustration / Figure

Here's my architecture diagram, as shown in Figure 1.

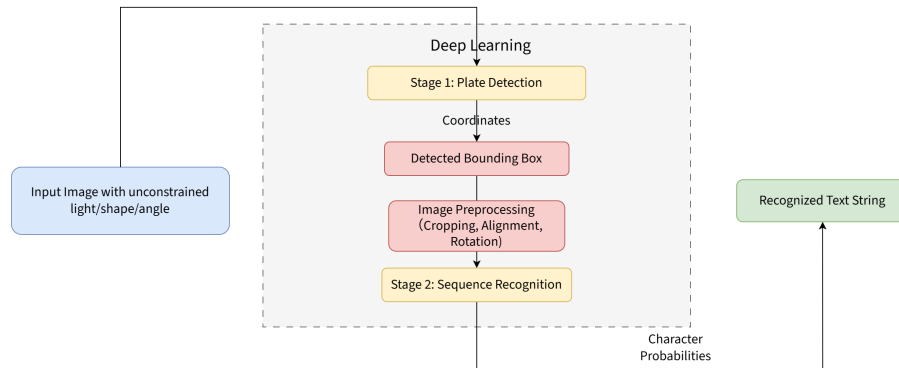


Figure 1: Architecture Diagram

Background & Related Work

OpenALPR [2] is the most well-known solution I've searched, it offers robust commercial options, but these are often for general-purpose use. On the non-commercial side, recent advancements focus on two distinct stages: detection and recognition. For detection, YOLO [3][4] architectures are widely adopted due to their real-time inference speeds, while specialized models like WPOD-NET [5] address the challenge of rectifying distorted plates captured at oblique angles. For the recognition stage, CRNN [6] have replaced traditional template matching, allowing for sequence learning without character-level segmentation. LPRNet [7] is another special option: it is a lightweight and fast model often deployed on embedded devices with limited computational capability. All of the above non-commercial models have numerous successful open-source demos, sufficiently demonstrating the project's feasibility.

Data Processing

First, I've generated 10k synthetic Ontario plates based on an HTML template, used the `jinja2` library in Python to replace the text with randomly generated 4 letters + 3-digit license plate numbers, and `playwright` library to print the plate into an image. Then, I applied Domain Randomization by adding random effects of geometric transformations, colour and saturation deviation, blur and pixel dropout to all these generated clean plate images, using `albumentations` library. This simulates the condition of license plates under various lighting conditions and levels of damage and dirt. After that, all of these images of aged license plates are composited into 30 images of the front and rear of real cars. The edges of the composites are blended using blur effects, and the backgrounds are slightly rotated and colour-corrected to complete the final data generation. The license plates generated will be labelled via their filenames in the format `"coordinates+plate\_number.jpg"`. This structure allows for easy extraction of the necessary label data during the training process.

Architecture

First, for the plate detection stage, I've designed and trained a custom Convolutional Neural Network to act as a bounding box regressor. It will utilize convolutional layers to extract spatial features, followed by fully connected layers that output a vector of four coordinates (x, y, z) , representing the precise location of the plate. Second, for the core part of the project, the text recognition stage, I will train a custom Convolutional Recurrent Neural Network from scratch. This network will consist of a CNN layer to extract visual features from the cropped plate, an RNN layer to learn the sequential dependencies of characters, and a post-processing layer to combine characters into a String and filter the illegal characters, ensuring that the output is a valid license plate number. To ensure that the model trained in the second stage can perfectly process the data output by the first stage, all training and validation data for the second stage are derived entirely from the inference output of the first stage.

Baseline Model

To evaluate the improvements offered by the proposed deep learning architecture, I compared it against a traditional computer vision baseline. This baseline will be implemented using OpenCV for detection and the Tesseract OCR engine for recognition. The baseline will operate in this pipeline:

The input image is first converted to grayscale and subjected to a blur function to reduce noise. I will then apply edge detection functions to identify structural boundaries, followed by contour recognition. Then crop the image to a rectangular shape and an aspect ratio characteristic of Ontario plates, and use Tesseract (an OCR engine that relies on pattern matching) to recognize characters in the picture.

Quantitative Results

The training and testing statistics are shown in Table 1 below, while the loss plot of the training is shown in Figure 1. The final validation loss and train loss have reached a well level ($< 10\%$). Both losses are low, which means that the model not only memorized the images, but also have the ability to infer the correct plate number from all these kinds of images. The testing set is a bit higher, which has 16.5% loss, the reason for this will be analyzed in the following parts.

Qualitative Results

Here are 2 example outputs of my model, as shown in Figure 2 and Figure 3. The green bounding box shows the visualization of the output coordinates from the bbox CNN stage, and the output characters are the output of the inference of the recognition CRNN stage, both include the correct license plates numbers, which suggests that the model performs well in qualitative results.

For privacy reasons, all data examples in this report use synthetic license plate images (created by combining generated license plates with photos of real vehicles) and do not contain any actual license plate numbers.

Total Epochs	50
Training Set Size	10,000
Validation Set Size	500
Test Set Size	500
Final Train Loss	0.0433
Final Val Loss	0.0883
Best Val Loss	0.0839
Test Loss	0.1653

Table 1: Training and Testing Statistics



Figure 2: Loss Plot



Figure 3: Prediction Result 1



Figure 4: Prediction Result 2

Evaluation of Model on New Data

To ensure the results accurately reflect the model’s true performance and generalizability, I evaluated the final architecture on a completely independent test dataset of real-world images. After finalizing the model’s hyperparameters and freezing the training process, I manually captured 30 entirely new photos of Ontario license plates across Bay Street and passed them through a simple domain randomizer filter to generate a 500-picture test dataset. These images were strictly sequestered: they were never passed through the synthetic domain randomization pipeline, never examined to influence the tuning of hyperparameters, and never used during the training or validation phases.

When evaluated on this completely unseen real-world data, the model achieved an 83.5% correctness rate (calculated by 0.1653 test loss as shown in Table 1) based on characters (this means based on character count rather than the entire license plate number), with a rate of 64.5% completely correct license plate numbers. This is lower than the performance on the synthetic validation set, which is expected when transitioning from synthetic training data to the unpredictability of real-world noise. The model successfully detected and read most of the characters on the plates under real camera-captured photos. However, since the license plate numbers predicted by the model often contain extra characters, the current model is unable to determine which characters are actually

redundant. As a result, it sometimes incorrectly truncates valid characters and retains invalid ones, leading to a very low accuracy rate for complete license plate number predictions.

In general, the model performance meets my expectations, as it can handle real-world data with only a few manual adjustments to get the completely correct plate number, which is undoubtedly a model with practical application value.

Result Discussion

Looking at the loss of the test set, as there is a 16.5% probability of making mistakes, the model actually has the ability to locate license plates and recognize the numbers on them. There is no doubt that the data output in qualitative testing is usable. However, when evaluated against a commercial-grade LPR system, this model is clearly subpar. For example, if using this model in a parking lot, an error rate of 16.5% means that 15 out of every 100 users will be wrongly charged due to a recognition error, which could result in significant legal and reputational damage. Moreover, since the background images in all validation and testing datasets were primarily captured outdoors on sunny days using high-resolution smartphone cameras, even though a domain randomizer was applied to the license plates during data generation, the lighting, angles, blurriness, and noise still lack realism and representativeness. In general, if used solely in a system designed to assist humans in quickly entering license plate numbers, this model performs very well and can save a significant amount of time, but if used as a standalone system capable of directly charging users or even issuing tickets, there is still a need for improvement to meet requirements for high availability, reliability, and compatibility.

Ethical Considerations

The collection of the validation dataset (real photos taken with a smartphone camera) poses privacy risks involving the capture of personal privacy beyond the license plate, such as drivers' faces or passersby. To mitigate this, strict data anonymization protocols must be applied for this project: faces and background landmarks that could identify private residences will be manually screened and blurred from all images. Also, automatic License Plate Recognition is a tool of mass surveillance: this model would record information about every individual passing by the camera. Any leakage of such information would expose the parties involved to legal risks. Additionally, the probabilistic nature of deep learning determines that the model won't be 100% correct. A wrong recognition could result in financial or legal harm to an innocent vehicle owner. Therefore, any real-world deployment would require much more verification and certification.

Project Difficulty / Quality

The difficulty of this project significantly exceeds standard baseline expectations due to both the complexity of the data generation pipeline and the custom dual-stage deep learning architecture. Because acquiring a large, labeled dataset of real license plates presents significant privacy risks, I had to engineer a custom data generation pipeline to create 10,000 synthetic Ontario plates. To ensure the model could generalize to real-world conditions, I implemented Domain Randomization—applying geometric transformations, colour deviations, blur, and artificial dirt—before carefully compositing them onto images of real vehicles. Architecturally, rather than simply fine-tuning pre-existing, off-the-shelf models, I designed and trained two custom networks from scratch: a Convolutional Neural Network (CNN) for bounding box regression and a Convolutional Recurrent Neural Network (CRNN) for complex sequence recognition. Connecting these two stages by implementing a robust Python pipeline script so that the CRNN training data was derived entirely from the inference output of the CNN added an advanced level of engineering complexity. Given this high level of difficulty and the lack of reliance on pre-built commercial tools, the model performs well, achieving a final validation loss of less than 10%, a testing loss at about 16.5% and demonstrating highly accurate qualitative predictions across varying simulated conditions.

References

- [1] *U of T License Plate Recognition (LPR)*. Available from: <https://www.utm.utoronto.ca/parking/license-plate-recognition-lpr>
- [2] *The Best License Plate Recognition Software* . Available from: <https://www.goaccess.com/blog/the-best-license-plate-recognition-software>
- [3] Abirami Vina, C. (2024). *Using Ultralytics YOLO11 for automatic number plate recognition*. Available from: <https://www.ultralytics.com/blog/using-ultralytics-yolo11-for-automatic-number-plate-recognition>
- [4] Muhammad Zeerak Khan, C. (2023). *Automatic Number Plate Recognition YOLO v8*. Available from: <https://github.com/Muhammad-Zeerak-Khan/Automatic-License-Plate-Recognition-using-YOLOv8>
- [5] Nguyen Dinh Tra, Nguyen Cong Tri, Phan Duy Hung, C. (2022). *Improving Warped Planar Object Detection Network For Automatic License Plate Recognition*. Available from: <https://arxiv.org/abs/2212.07066>
- [6] Author, C. (2025). *Manual Entry Title*. Available from: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453698/>
- [7] Dianwei Chi, Zehao Jia, Lizhen Liu, C. (2022). *Research on license plate recognition based on graphically supervised signal-assisted training*. Available from: https://github.com/LorenzoLamberti94/license-plates-ocr/blob/master/lp_recognition/README.md
- [8] Zhenbo Xu, C. (2022). *CCPD (Chinese City Parking Dataset, ECCV)*. Available from: https://github.com/detectRecog/CCPD/blob/master/Zhenbo_Xu_Towards_End-to-End_License_ECCV_2018_paper.pdf